PITFALL: Catching Point-in-Time Violations in Multi-Table Feature Pipelines by Differential Execution

First Author, Second Author, Third Author
ITMO University, St. Petersburg, Russia
{author1, author2, author3}@itmo.ru

***Abstract*—Features for multi-table prediction tasks are built by aggregating neighbouring tables at a per-row seed time. Correctness requires every aggregate to respect that seed time along every join path and for every column, including columns written to a row after the row appears. We find violations of this requirement in five sources we did not construct: a feature-engineering library called as in its own introductory example, our own reference code, code generated by two language models, published expert SQL from a benchmark's user study (8 of the 14 files that execute), and SQL written by an LLM agent. Inflation reaches 16 AUC points, but it is decided by the task rather than by the defect: our largest violation, hundreds of thousands of diverging cells, costs nothing measurable, and whether a one-character boundary error matters is settled by timestamp granularity. We present PITFALL, which runs a feature program twice, on the full database and on one truncated at the seed time, and reports any divergence. The check does not parse the program, has no false positives by construction, names the offending columns in seconds, localises the leak by truncating one availability channel at a time, and returns verdicts on a 37-query corpus in 19 minutes. The detector commercial AutoML platforms deploy against leakage, the highest AUC any single feature reaches, fails in both directions: at DataRobot's default threshold it is silent on every violation we report, and at H2O's it fires on correct pipelines.**

***Index Terms*—data leakage, point-in-time correctness, relational machine learning, AutoML, feature engineering, LLM-generated code**

I. Introduction

In multi-table (relational) machine learning a prediction task is given as a table of triples *(entity, seed time, label)* over a database with foreign keys and timestamps [8]. Predictive features are obtained by walking join paths out of the entity and aggregating neighbouring rows. Every aggregate may only read facts that existed at the seed time of the row it is computed for.

Leakage is a documented driver of irreproducibility in applied ML [1], [2], and this requirement is easy to violate: truncation is per row, so cutting the database once at the start of the test period says nothing about individual seed times; the requirement must hold along every join path, where the governing timestamp often belongs to a neighbouring table; and a row can carry columns written later than the row itself, such as a review attached to an order, so filtering by the order timestamp admits the row together with its future columns. This third case is the one most often missed.

Existing safeguards do not cover this case. Commercial AutoML platforms (DataRobot, H2O Driverless AI, SageMaker) all ship one detector, which we call the *univariate probe*: fit a model on each feature alone, take the highest AUC any single feature reaches, and flag the data when it exceeds a threshold, Gini Norm 0.85 (AUC 0.925) for DataRobot and AUC 0.80 for H2O [6], [7]. A leak carried by one feature is therefore caught, and the rest of this paper asks what happens when it is not. Static analyzers for notebooks [3] and pipeline instrumentation such as `mlinspect` [4] target preprocessing order and train/test contamination, not temporal leakage; asking a language model directly reaches 0.19 precision at 0.52 recall [5].

Contributions. (i) A simple execution-based check for feature programs, in the spirit of metamorphic testing: sound, one-sided, language-agnostic, extended with per-column availability and leak localisation by per-channel truncation. (ii) Evidence from five independent sources on six public databases (Section V-C), including published expert SQL run under the check, and a demonstration that timestamp granularity rather than the code decides whether a boundary error costs anything. (iii) An execution-verified measurement of the rate at which two language models produce temporally incorrect feature code, by task construction and prompt guard. (iv) A working system and an interactive demonstration in which the checker fires where the univariate probe is silent, stays silent on correct code, and localises the leak.

II. Point-in-Time Correctness

A. Definition

Let D be a database, t a seed time and φ a feature program. Each row r has a row timestamp $\mathrm{time}(r)$; a column c of r may in addition have its own availability timestamp $\mathrm{avail}_c(r) \geq \mathrm{time}(r)$. Write $D|_t$ for the database in which rows with $\mathrm{time}(r) > t$ are removed and values with $\mathrm{avail}_c(r) > t$ are replaced by NULL. Then φ is *point-in-time correct at t* if and only if

$$\varphi(D, t) = \varphi(D|_t, t). \tag{1}$$

For a cutoff shift $\delta \geq 0$, $I(\delta)$ is the AUC of a *fixed* learner on features computed with cutoff $t + \delta$ minus its AUC at $\delta = 0$; the learner is fixed because in one published trajectory [9] swapping the boosting implementation on identical features moved AUC by 11.5 points.

B. Availability relation

Equation (1) generalises by replacing the time predicate with an *availability relation* $R(r, e, t)$, true when row r may be read when predicting for entity e at time t; co-emission and group leakage are other masks over the same machinery. Availability is defined per column: in our data `review_score` becomes available at `review_creation_date`, a median of 10 days (maximum 147) after the order row it is attached to. Some columns have no availability timestamp at all, e.g. a mutable status field kept without history; the truncated database is then identical to the full one and no execution-based check can decide them (we exclude `order_status` throughout). The class is easy to under-count: a negative control revealed a second instance, the derived `late` flag of never-delivered orders, reading "not late" rather than "not yet known".

Why univariate probes miss this. A leaked signal spread over d aggregates loses visibility to any single-feature statistic as d grows, while its effect on a multivariate model does not fall.

C. Differential execution and localisation

The check follows from (1): run φ twice and compare:

```
program(db,t,ents) !=
program(db.truncate(t),t,ents)
```

A divergence proves violation: the same program, given strictly less information, produced a different answer, whether through an aggregate, a join, or a statistic fitted on the full table; there are no false positives by construction. The guarantee is one-sided (a violation that does not manifest at the tested seed time is not observed) and the program must be deterministic.

The same primitive localises the leak. The database has a finite set of *availability channels*: rows of table T appearing after t, and values of column c becoming known after t. Truncating one channel at a time identifies the channels on which the output changes and the output columns each affects (seven runs in our database); masking exactly those channels and re-checking confirms that no other channel contributes. Masking diagnoses without repairing, since any program passes on a pre-truncated input.

Why check rather than always truncate? With one seed time per call, truncating the input is itself a fix; but in training tables each row has its own seed time, so truncation must happen per row inside the program, which is what the check verifies, and a defect in code that will be reused should be reported, not masked once.

III. System

PITFALL sits between any feature builder and any tabular AutoML. SCOUT holds the availability map: which column governs the availability of which field, and which fields have none. Today it is a small hand-written declaration, which an LLM may propose from the schema, a question with an independently checkable answer unlike "is there leakage here" [5]. BUILDER is any producer of a feature program, library, agent or person. GUARD runs the differential check and returns the verdict with the diverging columns and cells; on a violation LOCATOR runs the per-channel truncation. Clean programs are handed to an unmodified tabular AutoML such as AutoGluon [12]. The core is ∼170 lines of Python over pandas, independent of the builder's language.

Validation. On six reference programs with known verdicts at three seed times the checker was right on 18 of 18 combinations. A negative control with the seed time set past every timestamp caught three defects of our own before the measurements that depended on them, and surfaced the second undecidable class of Section II-B. On rel-amazon it fired on the same files the checker was calling violations, because truncation changes the query plan and floating-point summation is not associative; resetting the tolerance from the control's own noise floor (1.7×10^{-11} relative) makes all four files clean. Four leak verdicts were false, and no reading of the checked code would have found it.

IV. Data and Setup

Olist [13] is a public Brazilian marketplace database, 7 tables and 112,650 order line items, on which we define three tasks in RelBench [8] form: seller activity (A), seller review quality (B) and product demand (C), at three seed times in 2018 with a 90-day horizon. Two columns arrive after the row they belong to, the review score and the delivery outcome.

rel-f1 [8] is the Ergast Formula 1 database, 9 tables and 26,080 driver-race results over 1950–2023. Our task is *will this driver retire from the next race*, seed time at midnight of race day. Race stamps change granularity inside this one database, which lets a boundary error be priced twice under otherwise identical conditions (Section V-B).

Corpus databases. These two carry every measurement of cost. The corpora of Section V-C additionally run against the databases they were written for, rel-f1 (tasks *driver-dnf*, *-position*, *-top3*), rel-stack, rel-event, rel-hm and rel-amazon: 15 tasks, tables of up to 10^8 rows, availability from RelBench's `time_col` declarations.

Model and intervals. One LightGBM configuration with a fixed seed is used everywhere, so differences come from the features alone. Intervals are paired percentile bootstraps of the AUC *difference* on the same test rows (2,000 resamples).

V. Evidence

A. The cost of a shifted cutoff

$I(\delta)$ increases monotonically over $\delta \in [0, 90]$ days on Olist tasks A and B at all three seed times (Fig. 1). On task C, where the cutoff is controlled separately for the entity's own history and for aggregates reached through a join to the seller, leaking only through the join costs $+3.0$ to $+5.1$ points and is invisible to the probe (Section V-D).

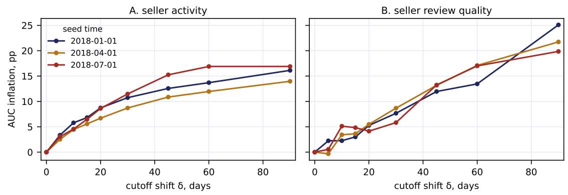

Fig. 1. Inflation $I(\delta)$ as the cutoff moves δ days past the seed time (Olist). A one-month error costs 5.8–11.5 AUC points; no cutoff at all, up to $+27.0$ (A) and $+18.0$ (B).

B. When an off-by-one costs everything and when it costs nothing

Whether a boundary error matters is decided by the timestamps, not by the code, and rel-f1 settles this inside one database: race stamps carry no time of day before 2005 and carry the start time from 2005 on. An outcome is available strictly after the race starts, so the inclusive cutoff $\leq t$ admits the race being predicted in the first era and nothing in the second. In 0.02 s per seed time, before any model is fitted, the checker diverges on every day-stamped seed time and is clean on every second-stamped one. The same one-character error then costs -0.45, $+2.45$ and $+3.27$ points on the day-stamped cells and exactly 0.00 on all three second-stamped cells. On Olist, where events carry seconds, this error also cost 0.00; without rel-f1 we could not have told whether that generalises.

Leaking only through the join path replicates here at $+0.72$ to $+1.49$ points. Removing the cutoff altogether costs -1.9 to -7.0 points, because aggregating a driver's whole career destroys the recency structure the task relies on.

C. Sources of violation

Five sources, none constructed by us; the univariate probe stays below DataRobot's warning threshold on every one. **A library default.** `featuretools` [10] 1.31.0 with `ft.dfs` and no cutoff-time table, the form of its introductory example [11], supports cutoff times but does not indicate that they are needed: 11 columns diverge at all three seed times, worth 14.8–16.3 points.

Our own reference code. It filtered history by order time and took all columns of the admitted rows, including the two carrying their own later timestamps; the checker flagged exactly the four review and delivery aggregates, worth 0.2 points on task A and 3.1–5.3 on task B.

Published expert SQL, executed. In the hand-written feature SQL of the RelBench user study [8], 15 files over 172 joins, a manual audit had found two violations and six of eight scanner candidates false. Each file is a DBT/Jinja template over DuckDB, run unchanged. Fourteen files execute; **8 diverge** and 6 are clean at every seed time, the latter under truncations removing up to $2{\times}10^7$ rows. The three rel-f1 files read `races` rows dated after the seed time; `stack/user-badge` takes a global frequency over `badges` with no cutoff, the violation the audit did find, where the checker names nine affected columns against its two; on `stack/post-votes`, which

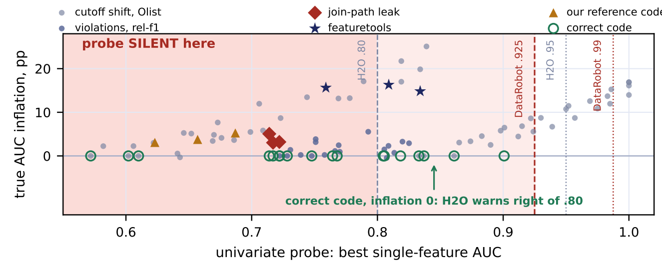

Fig. 2. The univariate probe, the highest AUC any single feature reaches (x), against the true inflation of the model (y), one point per run. The shaded region is silent at every deployed threshold yet holds violations worth up to 17 points, while correct pipelines, at inflation 0 by construction, extend past H2O's notification line.

the audit cleared, the output depends on 2,730 `users` rows dated after the posts they own. The file that does not execute reads `posts.AcceptedAnswerId`, a mutable field with no history that RelBench itself deletes with the comment *remove time leakage columns*: an independent party reaching our Section II-B verdict on the same column.

How much a clean verdict is worth. The three rel-event files join `users` without a cutoff, so six demographic columns read rows of users who registered after the seed time. The violation reaches 87.5% of label rows and is present at 23 of the task's 24 seed times; it is absent at exactly one, which is RelBench's own test seed time. Training labels are contaminated almost everywhere and test labels are clean, so train and test are built by different rules. The date is not a coincidence: joining in the last two weeks leaves no future window and earns no label, so no labelled user registers in the final fortnight. Data beyond a seed time only shrinks as that seed time grows, which makes a benchmark's test seed the least sensitive one available. Seed times must come from the middle of the training split rather than from the split the benchmark scores on.

An agent-written corpus. The 37 distinct queries an LLM agent wrote across 32 trials on `stack/user-engagement` had only a static scan: 5 candidates, no executed verdict. Running them takes 19 minutes: 34 execute, of which **2 diverge**; the rest exceed a 300 s budget or name a table that does not exist. Both violations are the same missing cutoff on the join to `users`, giving account ages down to $-2{,}021$ days on 57 of 25,233 label rows, and the query carrying it appears in 31 of the 32 trials. Against execution the scan errs both ways: 3 of its 5 candidates are false and 2 real, correcting our own earlier reading of that corpus.

D. What the univariate probe reports

No value of the probe separates leaking runs from clean ones (Fig. 2). Leakage through a join moves it not at all: the probe returns the same 0.71 with and without the leak while the model gains 3.0 to 5.1 points. On task B nothing reaches DataRobot's threshold, not even the complete absence of a cutoff, worth $+18.0$ points. H2O's lower threshold errs the other way, firing on the correct task-A pipeline at every seed

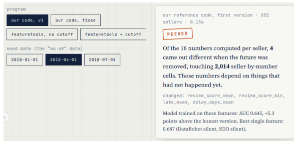

Fig. 3. The demonstration page: program and seed-date selectors, and the verdict with a plain-language summary. Below it, not shown, the outputs side by side.

time, because recency is a legitimate predictor of churn. Both errors replicate on rel-f1, where the probe moves by at most 0.015, and on the executed expert SQL, where it is identical to six significant figures in both regimes.

E. Generated feature code

We used two code models, `deepseek-v4-flash` and `bytedance-seed/seed-2.0-code`, to generate a Python function computing features for task C, giving the schema and the seed-time column, not mentioning "leakage" or "point-in-time". Verdicts come from the checker, with five flagged programs manually confirmed. 233 generations, 132 functional, 61 incorrect: 62% for the first model and 22% for the second. Roughly half the generations crash before producing a feature table, a share that does not fall as the sample grows, so the rate among those that run needs a bound: over all 233 generations it lies between 26% and 70%. Models fail the same, filtering the entity's own history by the primary event timestamp, ignoring the later timestamp of the joined one. We make no claim about models in general. Task construction is decisive: on a second task with one seed time per entity and no secondary time axis, both models produce 0 violations. Prompt guards help partially but not to zero: a reminder and then a per-aggregate requirement cut the pooled rate from 46.2% to 14.6%.

F. Cost of a violation is task-dependent

The same defect in the same code costs 3.09 to 5.26 points on task B, while on task A every interval contains zero (0.16 to 0.32); among model-generated programs the median inflation is 0.11 points. The expert SQL makes the same point on code we did not write: rebuilding its features on a per-row truncated database and refitting the same model, the violations cost -0.33 on `f1/driver-dnf`, $+0.95$ on `f1/driver-top3` and -0.02 on `stack/user-badge`, the last despite more than 2×10^5 diverging cells. Neither the fact of a violation nor its size predicts its cost.

VI. Demonstration

The demonstration is a self-contained interactive page built from live runs (Fig. 3) plus a console runner, `python3 demo.py`. A visitor drags the seed date over one seller's real orders and watches a naïve "average review" feature change, compares programs side by side, moves the cutoff and the probe threshold, and submits a feature function of their own. The console scenes replay Section V-C live, ending in per-channel localisation of each diverging column.

VII. Limitations

(i) One-sided. The check proves violation, not correctness, and only with respect to the observed database, the chosen mask and the seed times tested; the rel-event case above shows that gap is not hypothetical. *(ii) Undecidable columns.* Mutable fields kept without history cannot be checked by execution. *(iii) The availability map.* A wrong timestamp means the wrong thing is checked, and the map is a modelling decision: RelBench's `time_col` places the rel-f1 race calendar after the seed time, though a practitioner may consider it published in advance; the check decides code against a declared relation rather than against intent. *(iv) Scope.* Cost is measured on two databases and four tasks only; the corpus verdicts span five more databases but carry no cost. The expert-SQL corpus executes on 14 of 15 files, and audit and checker disagree on 7 of those 14, always with the checker finding more. *(v) Determinism.* Columns that differ between identical reruns carry no verdict, and one rerun cannot prove that a column is deterministic, so cell counts move between runs while file verdicts do not. *(vi) Final-program metric undercounts:* validation-driven selection can discard a leaking intermediate trial (observed once).

VIII. Conclusion

Point-in-time violations occur in library defaults, in published expert code and in generated code, and the univariate probe used to catch them misses the regimes reported here. Their cost is not predictable from their existence or size, so correctness must be tested rather than scored; differential execution decides it in seconds and localises the leak with the same primitive. A clean verdict is only as strong as the seed times it was taken at.

Data, Ethics and Availability

Experiments use public datasets: Olist (Kaggle, CC BY-NC-SA 4.0, anonymised by its publisher) and the rel-f1, rel-stack, rel-event, rel-hm and rel-amazon databases with the expert SQL released alongside RelBench [8]; no human subjects. Language-model outputs are released unedited. Code, data and all reported numbers: https://github.com/stas1f1/pitfall.

References

[1] S. Kaufman, S. Rosset, and C. Perlich, "Leakage in data mining: formulation, detection, and avoidance," in *Proc. KDD*, 2011.
[2] S. Kapoor and A. Narayanan, "Leakage and the reproducibility crisis in machine-learning-based science," *Patterns*, vol. 4, no. 9, 2023.
[3] C. Yang *et al.*, "Data leakage in notebooks," in *Proc. ASE*, 2022.
[4] S. Grafberger *et al.*, "mlinspect: a data distribution debugger for ML pipelines," in *Proc. SIGMOD*, 2021.
[5] N. Alturayeif and J. Hassine, "Data leakage detection in ML code," *PeerJ Computer Science*, 11:e2730, 2025.
[6] DataRobot documentation, "Data quality assessment: target leakage," docs.datarobot.com, 2026.
[7] H2O.ai, Driverless AI 1.11 User Guide, "Data leakage and shift detection," docs.h2o.ai, 2026.
[8] J. Robinson *et al.*, "RelBench: a benchmark for deep learning on relational databases," *NeurIPS D&B*, 2024.
[9] "RelAgent: agentic feature engineering for relational data," arXiv:2605.07840, 2026.
[10] J. M. Kanter and K. Veeramachaneni, "Deep feature synthesis," in *Proc. DSAA*, 2015.
[11] Featuretools 1.31 documentation, "Handling time," featuretools.alteryx.com, 2026.
[12] N. Erickson *et al.*, "AutoGluon-Tabular," arXiv:2003.06505, 2020.
[13] Olist, "Brazilian e-commerce public dataset," Kaggle, 2018.